

UT Workflow 双通道 Skill 设计

Date: 2026-06-18 (revised 2026-06-19 after grilling session, v5)

Status: Design proposal (v5 — process model clarified)

Scope: hermes_workflow skill 新建 + ut/workflow skill 更新 + ut/workflow_loop_core skill 新建 (共享循环主体) + Worker SKILL.md 更新 + hermes_runner.py 重构 + ut-supervisor profile 新建 + 3 Gateway profile 部署文档

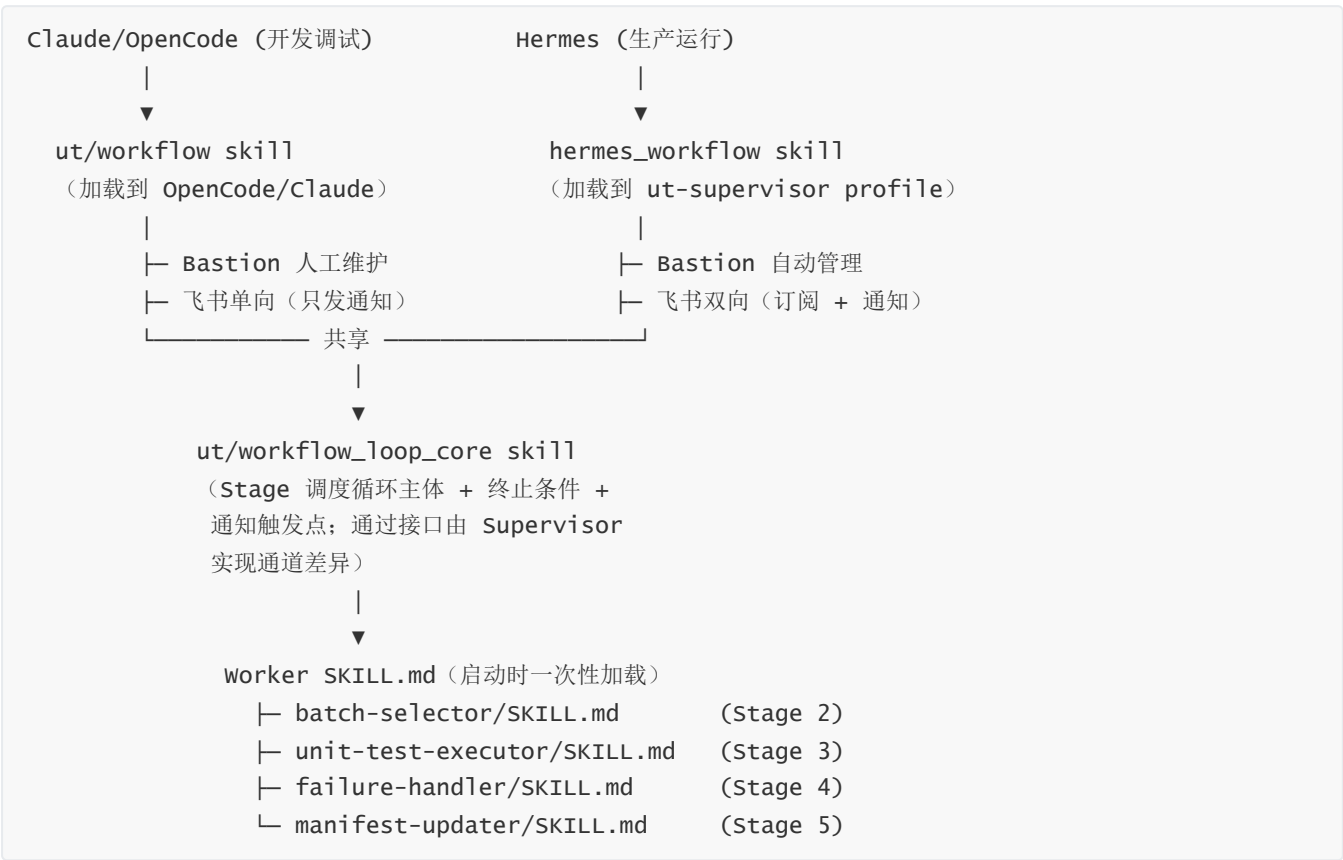
1. 背景

当前 UT Workflow 只有 ut/workflow 一个 skill，通过 Claude Code/OpenCode Agent 加载。该模式下 Bastion daemon 依赖人工维护，不适合长时间无人值守运行。

[2026-06-18 设计文档](#) 已确定 Hermes 应作为生产级运行主体。本设计在此基础上细化双通道 skill 架构与进程模型。

2. 核心结论

两个 Supervisor skill + 一个共享循环主体 skill + 同套 Worker + 一个 ut-supervisor profile + 3 个 Kanban Gateway profile:



ut/workflow_loop_core/SKILL.md 承载循环主体，避免两个 Supervisor 重复维护循环逻辑。Supervisor 只实现"通道差异接口"—— ut/workflow 实现"无指令检查 / 无 OTP"， hermes_workflow 实现"飞书指令 / OTP 流程"。

3. 两个 Skill 职责边界

	ut/workflow	hermes_workflow
触发方	Claude Code / OpenCode	ut-supervisor profile 内的 Hermes Agent
触发方式	终端用户输入关键词	飞书消息关键词匹配 / Hermes CLI
初始化	终端交互式问答	飞书参数确认卡片
Bastion daemon	人工维护（断联发告警卡片）	自动管理（心跳 + 飞书 OTP 恢复）
飞书方向	单向（只发通知卡片）	双向（订阅消息 + 发通知）
状态机	简化（无 paused/waiting_otp）	完整（5 状态 + pending_config）
适用阶段	开发调试	生产运行

4. Worker SKILL 加载模型

Stage 2-5 的 Worker SKILL.md 在 workflow 启动时由 Supervisor **一次性加载**到 context；循环中直接按 stage 名引用执行，**不重复加载**。

Fallback 机制：循环中若发现 SKILL 引用缺失（例如 auto-compact 把早期加载内容压缩掉了，或 context 被截断），Supervisor 应当**按需重新加载缺失的那一份 SKILL**，然后继续执行。Supervisor 不假设 SKILL 一定常驻。

优势：

- 1. 正常情况下 SKILL.md 内容只计费一次，循环 N 轮节省 $\sim(4 \times \text{skill_size} \times (N-1))$ token
- 2. SKILL 段成为稳定 prefix，prompt cache 命中率高
- 3. 循环逻辑简单，无需在每轮做 "load skill" 操作
- 4. 即便 harness 行为不可控（auto-compact 把 SKILL 段也压了），fallback 兜底，正确性不受影响

跨 Stage 数据传递：通过本地文件（manifest.json / batch_results.json / handled_tests.json），不在 Supervisor context 中累积保存。pytest stdout 等大块数据通过 §5.2 远端关键段提取，只传摘要回 Supervisor，进一步降低 context 增长速度。auto-compact 仅作为兜底机制处理超长情况。

ut/workflow_loop_core/SKILL.md 提供给 Supervisor 调用的接口（伪签名）：

```
loop_core.run(  
    stage_skills,                # {"stage2": <ref>, "stage3": <ref>, ...}  
    handle_checkpoint(state),    # Supervisor 实现：进度通知、指令检查  
    handle_bastion_disconnect(), # Supervisor 实现：ut 发告警卡片；hermes 走 OTP 流程  
    check_user_commands(state), # Supervisor 实现：ut 空实现；hermes 读飞书  
    check_terminal_conditions(), # 共享：pending_count==0 / consecutive_failures /  
    error_rate  
)
```

注：Kanban 模式下 ut-supervisor 不直接执行 Worker SKILL（Stage 逻辑在 Kanban Worker subprocess 里跑），但仍需要为 **Hermes CLI / 终端调试**保留一次性加载能力。

5. Stage 行为规范（两 Supervisor 共享）

5.1 Stage 2: select_batch

加载 `batch-selector/SKILL.md`。Worker 读 manifest，按以下规则筛选 batch：

```
selected = [
    t for t in tests if
        (t["status"] == "pending") or
        (t["status"] == "fixed_pending_verify") or
        (t["status"] == "retriable_error" and t.retry_count < t.max_retry) or
        (t["status"] == "failed" and t.retry_count < t.max_retry)
]
# 注意: error 状态测试不被选中（直接进 Stage 4）
```

5.2 Stage 3: execute

加载 `unit-test-executor/SKILL.md`。Worker 行为：

场景	Worker 行为
正常 PASS/FAIL	标记对应状态
OOM (CUDA out of memory)	标 <code>retriable_error</code> + <code>error_type=oom</code>
pytest 超时	标 <code>retriable_error</code> + <code>error_type=timeout</code>
collection error / import error	标 <code>error</code>
Bastion 断联 (agent.py run 返回连接错误)	调用 <code>bastion.mark_disconnected()</code> 上报，返回 <code>next_action=wait</code> ；不标记测试
Worker 不做任何重试	所有重试由 Supervisor 循环触发

关键段提取（远端只写 `raw_log`，本地写 `summary`）：

pytest 跑完后的处理流程：

1. 远端：pytest 重定向写 `{remote_batch_dir}/raw_log.txt`（only this file on remote）
2. Worker 调 `agent.py run` 远端执行 `grep -E 'FAILED|ERROR|PASSED' raw_log.txt + tail -N`，回传文本（≤200KB）
3. Worker 把回传文本本地落盘：`{local_batch_dir}/summary.txt`
4. 完整 `raw_log.txt` 留远端，仅 failure-handler 在 `summary` 信息不足时按需 `agent.py run "tail -N <raw_log_path>"` 拉片段

远端日志路径写入 `batch_results.json`（batch 层级，权威路径）：

```
// {local_batch_dir}/batch_results.json
{
    "batch_id": "batch_20260619_001",
    "remote_log": {
```

```

    "host": "t_h20",
    "container": "v0.13.0_torch2.5.1_compile",
    "raw_log_path": "/gpfs/.../ut_logs/<run_id>/batch_20260619_001/raw_log.txt",
    "size_bytes": 12345678,
    "captured_at": "2026-06-19T10:00:00Z"
  },
  "tests": [
    {"test_id": "test_a", "status": "failed", "error_type": "...", ...}
  ]
}

```

manifest.json 每个 test 加 `last_batch_id` 字段（指针，按需 resolve 到对应 batch_results.json 取 `remote_log`）：

```

// manifest.json tests[i]
{
  "test_id": "test_a",
  "status": "failed",
  "last_batch_id": "batch_20260619_001",
  "retry_count": 1,
  "max_retry": 3
}

```

failure-handler 看到失败测试 → 读 `last_batch_id` → 打开对应 `batch_results.json` 取 `remote_log.raw_log_path` → 按需 `agent.py run "tail -1000 <path>"` 拉完整日志片段。

5.3 Stage 4: handle_failures

加载 `failure-handler/SKILL.md`。Worker 行为：

- 处理范围：failed + error（不处理 `retriable_error`）
- 只读失败/错误测试的 summary（不读 PASS 测试日志；summary 不够时按需读远端 raw_log）
- 分类 C/E/D/P/M/S
- 原则：Agent 判断能修就修，重试验证，超过 `retry_count` 上限标记 ignored
- 不做行数/架构/补丁的人为限制 — Agent 自己判断边界

vLLM 源码自动修复约束：

- 修复必须 commit 到独立分支 `2.5.1_ut_verify`，**不允许 commit 到 master 或其他分支**
- failure-handler 启动前先校验：远端 vLLM repo 当前 HEAD 必须在 `2.5.1_ut_verify` 上，不在则报错退出（防止误改其他人的工作分支）
- commit message 用固定前缀 `[auto-fix]` 便于事后 `git log --grep="[auto-fix]"` 检索
- 完成时（`pending_count==0`）的飞书完成卡片附带“本次新增 **auto-fix commit 列表**”——`git log master..2.5.1_ut_verify --oneline` 摘要

C/E/D/P/M/S 处理细节：保留现有，详见 `failure-handler/SKILL.md`。

5.4 Stage 5: update_status

加载 `manifest-updater/SKILL.md`。Worker 行为：

- 合并 `batch_results.json` + `handled_tests.json` 到 `manifest.json`
- `retry_count` 累加
- 每个 test 的 `last_batch_id` 字段更新为本轮 `batch_id`
- **`retriable_error` + `retry_count` >= `max_retry` → 直接 ignored** (理由: "max retry exceeded for {error_type}")
- 重算 statistics

设计取舍说明：`retriable_error` 满次后**不进 failure-handler**——OOM/timeout 在生产环境出现概率较低，简单优先；继续迭代时如发现这类问题占比上升再补"针对性修复"路径。

6. 状态枚举 (manifest 测试 status)

状态	含义	谁产生	谁消费
<code>pending</code>	新测试	初始化	Stage 2 → Stage 3
<code>running</code>	正在跑	Stage 3 进行中	—
<code>passed</code>	通过 (终态)	Stage 3	—
<code>failed</code>	断言失败	Stage 3	Stage 2 (可重试) / Stage 4 修复
<code>error</code>	不可重试错误 (collection/import)	Stage 3	Stage 4 直接处理 (不进 Stage 2)
<code>retriable_error</code>	可重试错误 (OOM/timeout/网络)	Stage 3	Stage 2 (可重试, 不进 Stage 4)
<code>fixed_pending_verify</code>	修复后待验证	Stage 4	Stage 2 → Stage 3
<code>ignored</code>	放弃 (终态)	Stage 4/5	—

`error_type` 枚举新增 `oom` / `timeout`。

7. 通知规则

条件	事件	卡片颜色
每个 batch 完成	progress	 蓝色
<code>error_rate > 0.5</code>	alert	 红色 (继续跑)
<code>error_rate > 0.8</code>	paused	 黄色 (暂停等用户)

条件	事件	卡片颜色
<code>consecutive_failures > 50</code>	paused	■ 黄色（暂停等用户）
<code>stats.passed % 100 == 0</code>	milestone	■ 蓝色
用户主动暂停	paused	■ 黄色
<code>pending_count == 0</code>	complete	■ 绿色（附带 auto-fix commit 列表）
Bastion 断联 OTP 请求	otp_required	■ 红色
用户终止	stopped	○ 灰色
系统故障	failed	■ 红色
Gateway 任一挂	gateway_down	■ 红色（等 systemd 自动恢复）

OTP 渐进重发节奏（`hermes_workflow` 专属）：

第 1 次发卡片	→ 立即
未收到 → 第 2 次	→ 5 分钟后
未收到 → 第 3 次	→ 15 分钟后（@ 用户）
未收到 → 第 4 次	→ 30 分钟后（@ 用户）
未收到 → 第 5 次起	→ 每 60 分钟（@ 用户，封顶）

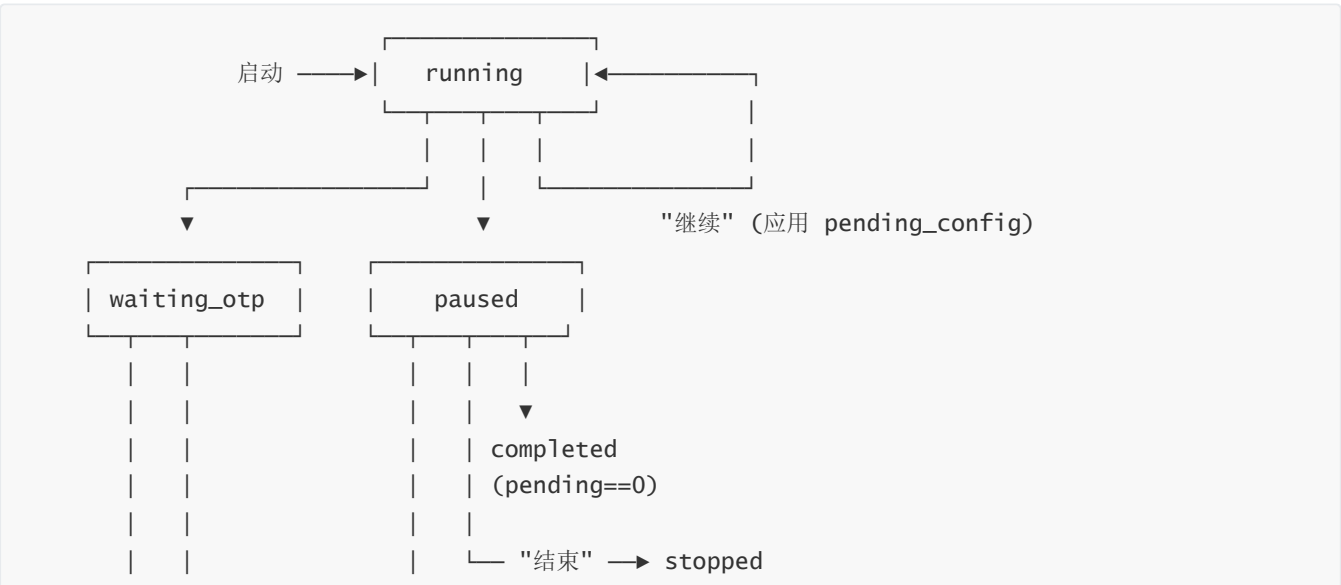
退出 `waiting_otp` 状态的方式只有两种：

- 1. 收到合法 OTP → 同状态内完成 daemon 重启 → `running`
- 2. 用户飞书发 "结束" → `stopped`

不再因超时进 `failed`。

8. 状态机（`hermes_workflow` 专属）

`ut/workflow` 不维护状态机（终端模式没有暂停恢复）。`hermes_workflow` 完整状态机：





关键变更：

- 删除 `reconnecting` 瞬态——OTP 收到后 daemon 重启在 `waiting_otp` 状态内同步完成
- `waiting_otp` 不再有"超时进 failed"路径

8.1 终态区分

终态	含义	触发
<code>completed</code>	正常完成	<code>pending_count == 0</code>
<code>stopped</code>	用户主动停止	用户指令 "结束" / "终止"
<code>failed</code>	系统故障	仅在初始化校验失败、配置无效等启动期错误

8.2 状态-指令矩阵

当前状态	"暂停"	"继续"	"结束"	"改参数 N"	"OTP code"
<code>running</code>	→ paused	忽略	→ stopped	→ paused (存 pending_config)	忽略
<code>paused</code>	忽略	→ running (应用 pending_config)	→ stopped	更新 pending_config	忽略
<code>waiting_otp</code>	忽略	忽略	→ stopped	忽略	同步重启 daemon → 成功 → running / 失 败 → 维持 waiting_otp
<code>stopped</code>	忽略	忽略	忽略	忽略	忽略
<code>completed</code>	忽略	忽略	忽略	忽略	忽略
<code>failed</code>	忽略	忽略	忽略	忽略	忽略

非指令消息，Agent 可回复但不改变状态。

8.3 指令优先级

`stop` > `pause` > `change_config` > `resume`

8.4 指令检查时机

每轮 Stage 5 后、发 progress 卡片之前。

设计取舍说明：

远端 pytest 一旦启动无法暂停（强杀风险高），且用户指令低频。等当前 batch 跑完再响应可以接受。

8.5 daemon 重启失败处理

OTP 收到但 daemon 重启失败 → **维持** `waiting_otp`，飞书改卡片为"daemon 重启连续失败 N 次，请人工介入"。不再 3 次失败进 `failed`。

9. resume 状态映射

旧状态	续跑行为
<code>running</code>	→ running，假设进程崩溃恢复，从 <code>current_stage</code> 继续
<code>paused</code>	应用 <code>pending_config</code> （如有） → running
<code>waiting_otp</code>	重新走 OTP 流程（渐进重发节奏从第 1 次开始）
<code>stopped</code>	拒绝续跑 ：飞书卡片"该 run 已被停止，请新建运行"
<code>completed</code>	拒绝续跑 ：飞书卡片"该 run 已完成，请新建运行"
<code>failed</code>	重新走启动流程：检查 daemon、配置等 → 重置状态为 running

10. pending_config 定义

用户说"改参数 key=value"时暂存到 `workflow_state.json`：

```
{
  "pending_config": {
    "batch_size": 4,
    "pytest_args": "-q --tb=short"
  }
}
```

白名单 key： `batch_size`， `pytest_args`， `max_retry_per_test`， `timeout`。其他 key 静默忽略并飞书提示。

规则	说明
只存白名单内的 key	其他 key 忽略
空对象 = 无待确认变更	<code>{}</code> 表示用户只暂停没改参数
不写回 workflow.yaml	临时修改只对本次有效
"继续"时应用	runner 合并到当前配置 → 清空 <code>pending_config</code>

规则	说明
"结束"时丢弃	不应用，直接清空
飞书 paused 卡片展示	列出 pending_config 内容供用户确认

11. hermes_runner.py 重构

11.1 当前问题

`hermes_runner.py` 内联重写了 Stage 2/3/4/5 逻辑，与已有脚本完全割裂，且功能更弱。

11.2 重构后定位

工具模块——被 `ut-supervisor profile` 内的 Hermes Agent / `ut/workflow_loop_core` `import` 使用。

```
# 初始化
init_or_resume(workflow_yaml_path, resume_from=None) → (run_dir, state_path, state)
read_state(state_path) → state
write_state(state_path, state)
read_manifest(manifest_path) → manifest

# Bastion (封装 BastionManager)
ensure_bastion(profile, feishu_config, state_path) → BastionManager

# 飞书
create_feishu(feishu_config_path) → FeishuAPI
send_card(feishu, event, manifest, iteration, **kwargs)

# 用户指令检查
check_commands(feishu, state_path) → list[Command]

# pending_config
apply_pending_config(state_path)

# 配置校验
validate_required_config(workflow_yaml) → (ok, missing_fields)

# 循环控制
check_stop_conditions(state_path) → (should_stop, reason, terminal_state)

# Kanban 模式 (Gateway 由 systemd 独立管理, 3 个 profile 各一实例)
check_gateways_alive() → dict[str, bool] # {profile_name: alive}
create_initial_kanban_task(workflow_yaml) → task_id # 创建首个 ut-orchestrator 任务
poll_kanban_stats(board_slug) → {pending, running, done}
```

11.3 不再内联 Stage 逻辑

删除 `stage_select_batch()` 等函数。需要调脚本时用 `subprocess` 调用 `execute_batch.py` 等已有脚本。

11.4 不再启动 Gateway

删除 `start_gateway` 调用——3 个 Gateway (`ut-orchestrator` / `ut-executor` / `ut-fixer`) 由 `systemd` template unit 独立管理 (详见 `docs/guides/hermes-gateway-service.md`) 。

`check_gateways_alive()` 在 workflow 启动前 + Kanban 主循环每轮都检查；任一 Gateway 挂 → 飞书发卡片 + 等 `systemd` 自动恢复 (不需用户介入) 。

12. batch_size 与 max_retry_per_test

12.1 batch_size

默认值 8 = GPU 数 (8) × 每 GPU 串行测试数 (1) 。

真实理由：

- 1. **日志大小**：单个失败测试 `traceback` + `tensor dump` 可能 5-10MB；batch 越大，失败时关键段提取压力越大
- 2. **OOM 风险**：每张 GPU 不能并行多个用例，所以 batch 内并行度上限 = GPU 数
- 3. **依赖 pytest 并行**：`gpu_scheduler.py` 已支持每 GPU 一个 `pytest` 进程

调整方向：验证 `pytest` 并行稳定后可调 16/32/64 (每 GPU 串行 2/4/8 个测试) 。

12.2 max_retry_per_test

默认值 3 (写在 `workflow.yaml`) 。

```
workflow.yaml:max_retry_per_test = 3    ← 全局默认
    ▼ 初始化时填到
manifest.json:tests[i].max_retry = 3    ← per-test 上限
manifest.json:tests[i].retry_count++    ← per-test 计数
```

13. 现有可复用 API 清单

模块	可复用 API	覆盖场景
<code>bastion_manager.py</code>	<code>mark_disconnected()</code> , <code>mark_connected()</code> , <code>requestotp()</code> , <code>start_heartbeat()</code> , <code>stop_heartbeat()</code>	Bastion OTP、心跳、断联恢复
<code>feishu_api.py</code>	<code>send_card()</code> , <code>send_message()</code> , <code>get_group_messages()</code>	飞书通知 + OTP 收信 + 用户指令接收
<code>send_progress_card.py</code>	命令行 <code>--event progress/complete/alert/paused</code>	所有飞书卡片场景

模块	可复用 API	覆盖场景
<code>init_workflow_state.py</code>	命令行 <code>--workflow-yaml</code> / <code>--resume-from</code>	初始化、续跑
<code>execute_batch.py</code>	<code>execute_batch(batch_config_path, workflow_state_path)</code>	pytest 执行
<code>gpu_scheduler.py</code>	<code>GPUScheduler</code> 类	GPU 检测分配 (每 GPU 一个 pytest 进程)
<code>retry_test.py</code>	<code>retry_with_delay()</code>	单测重跑
<code>classify_error.py</code>	错误模式正则	Stage 3 初步分类
<code>monitor_kanban.py</code>	命令行 <code>--workflow-yaml --poll-interval</code>	Kanban 模式监控

`start_gateway.py` 不再被 **workflow** 调用——3 个 Gateway 由 systemd 独立管理。

用户指令解析 ("暂停"/"继续"/"结束"/"改参数") 由 Agent 自然语言理解完成，OTP 码和 `key=value` 格式用正则。

14. 进程模型与部署架构

14.0 进程模型

`hermes_workflow` 模式涉及 **1 个 Hermes Agent profile (ut-supervisor, 长期) + 3 个 Hermes Gateway profile (ut-orchestrator/ut-executor/ut-fixer, 长期)**。两类 profile 用途不同：

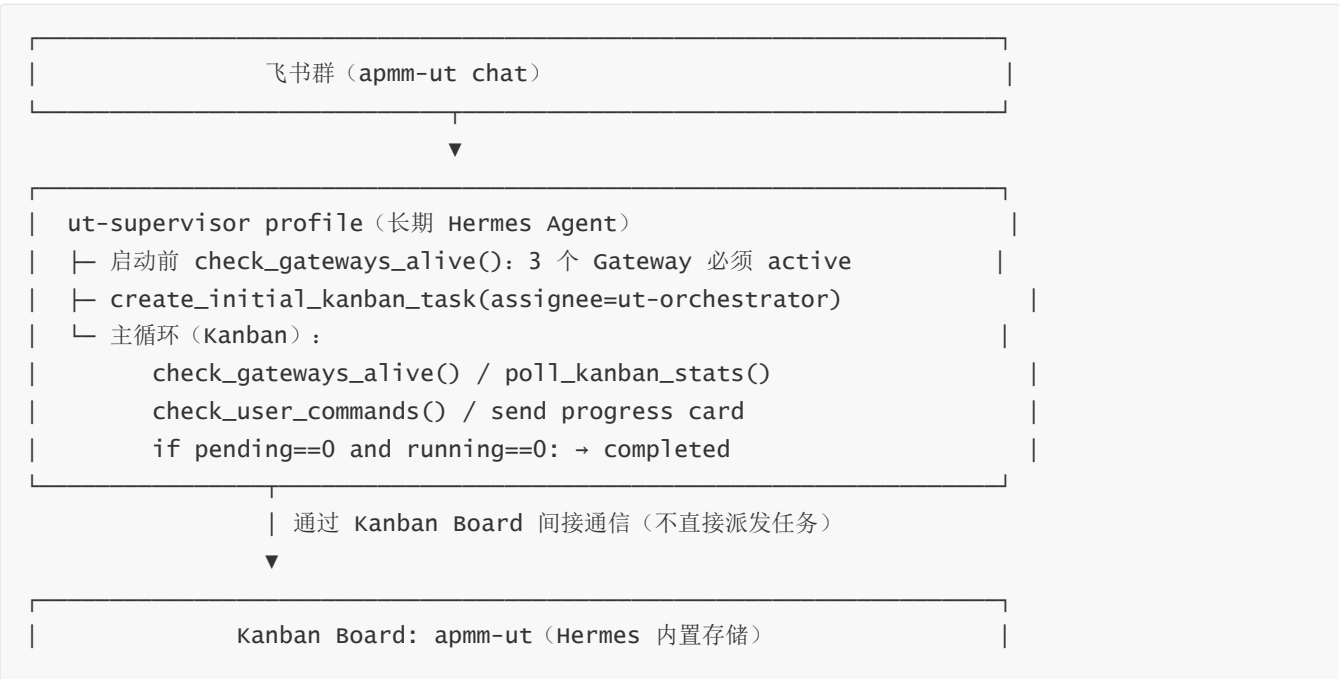
	ut-supervisor (1 个)	3 Kanban Gateway
进程类型	长期 Hermes Agent (订阅飞书)	长期 Hermes Gateway (dispatcher)
数量	1	3 (每个 worker profile 一个)
systemd unit	<code>hermes-agent@ut-supervisor</code>	<code>hermes-gateway@{ut-orchestrator,ut-executor,ut-fixer}</code>
加载内容	hermes_workflow + workflow_loop_core + 4 Worker SKILL	不加载 SKILL；只是派发器
响应飞书	✅ 是飞书订阅者	❌ 不订阅
管 workflow 状态机	✅ Supervisor 循环	❌
派发短任务	❌ 只创建初始 Kanban 任务	✅ 派发 Worker subprocess
运行时长	workflow 全程 (几小时-几天)	永久常驻 (独立于 workflow)
线性模式启用?	✅ 主体 (直接跑 Stage 2-5)	❌ 不启动

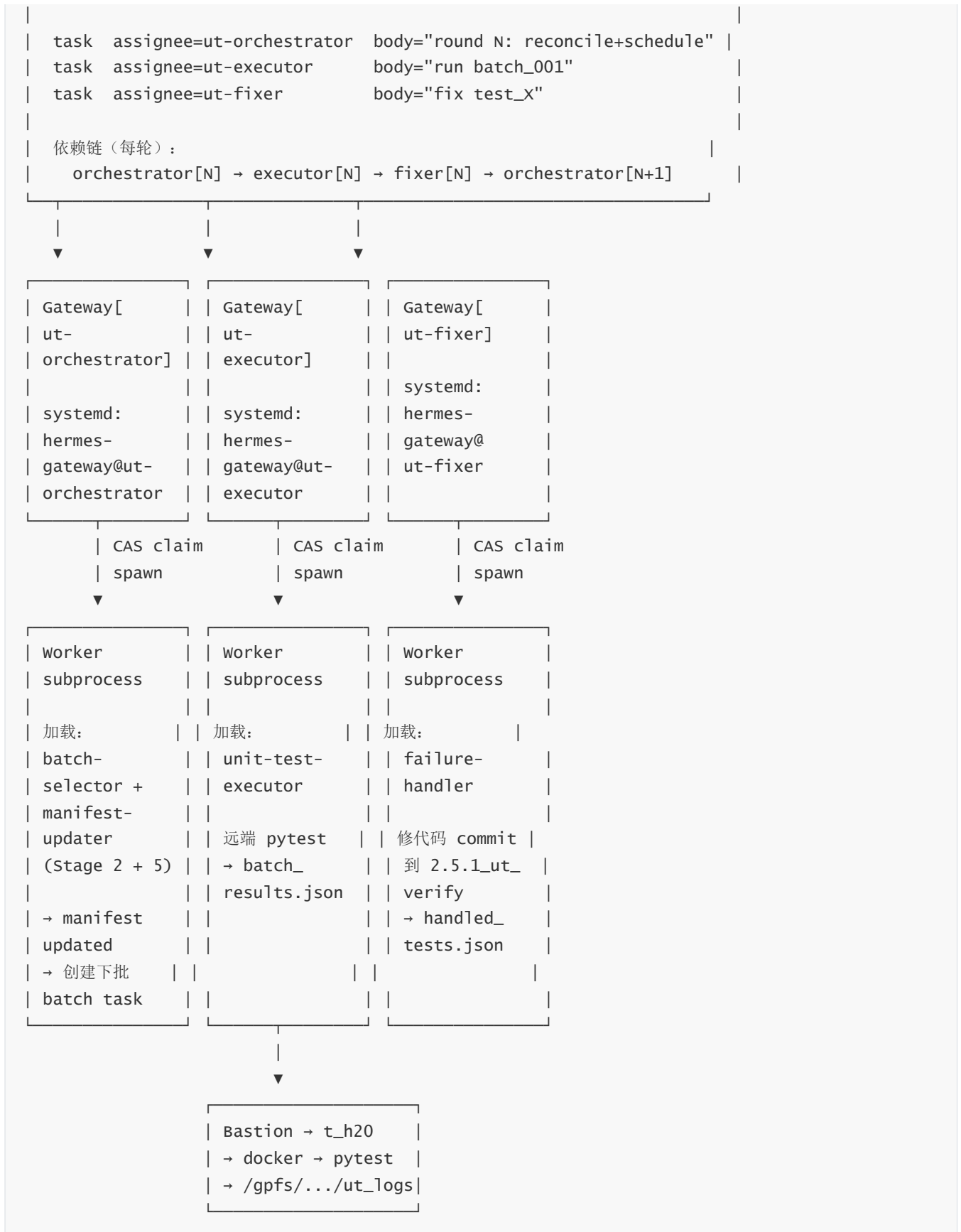
	ut-supervisor (1 个)	3 Kanban Gateway
Kanban 模式启用?	☒ 状态监控 + 飞书	☒ 派发 worker

交互图 1: 线性模式 (kanban.enabled: false)



交互图 2: Kanban 模式 (kanban.enabled: true)





关键：Kanban 模式下，**ut-orchestrator profile 的 Worker subprocess 同时承担 Stage 2 (batch-selector) 和 Stage 5 (manifest-updater)** ——它读上一轮 batch_results.json + handled_tests.json，更新 manifest.json，然后创建下一轮 batch task。这样 manifest 写者唯一（每轮只有 orchestrator subprocess 写），避免并发写冲突。

ut-supervisor 在 Kanban 模式下**不碰 manifest**，只做监控 + 飞书 + Bastion + 终止条件检查。

14.1 触发方式

通道	方式	示例
飞书消息	ut-supervisor profile 订阅飞书群， 关键词匹配	"跑 ut workflow"、"启动测试"、"开始 UT"
Hermes CLI	显式调用	<code>hermes skill run hermes_workflow</code> (连 ut-supervisor profile)

14.2 启动流程

1. 用户在飞书群发"跑 ut workflow"
 2. ut-supervisor profile 内的 Hermes Agent 收到消息（飞书订阅）
 3. 关键词匹配 → 加载 hermes_workflow/SKILL.md + workflow_loop_core/SKILL.md
+ 一次性加载 4 份 Worker SKILL.md
 4. Agent 飞书发【参数确认卡片】（蓝色）：
 - 展示 5 字段: test_list_path, batch_size, manifest_source, kanban.enabled, resume_from
 - 提供操作选项: "确认" / "yaml=PATH" / "resume=RUN_DIR" / "改 KEY=VALUE" / "取消"
 5. 等待用户回复（5 分钟超时 → 退出）
 6. 用户"确认" → 校验必备参数
 - test_list_path 或 manifest_source（至少一个，文件存在）
 - feishu_chat_id 存在
 - bastion.profile 在 .bastion_creds 中存在
 - kanban.enabled = true 时额外校验 3 个 Gateway profile systemd 服务全 active
 - 任一缺失 → 飞书发红色错误卡片 → 退出
 7. import hermes_runner, 调用 init_or_resume()
 8. ensure_bastion() → daemon 不可用 → 进入 waiting otp, 按 §7 渐进重发节奏发 OTP 卡片
 9. start_heartbeat()（心跳线程检测断联只调 mark_disconnected 上报，不直接弹卡片）
 10. 进入主循环（loop_core.run, 传入 stage_skills 引用 + 通道差异回调）

14.3 主循环

通过 `ut/workflow_loop_core/SKILL.md` 提供的 `loop_core.run(...)`, Supervisor 实现回调即可。

线性模式 (`kanban.enabled: false`) :

```
state = "running"
while True:
    # Stage 2-5 (按 stage_skills 引用执行, 引用缺失则按需 reload)
    执行 batch-selector → batch_config.json
    执行 unit-test-executor → 远端 raw_log + 本地 summary + batch_results.json
    (worker 检测到 Bastion 错误 → 调 bastion.mark_disconnected() 上报)
    执行 failure-handler → handled_tests.json
    执行 manifest-updater → manifest.json updated (含 last_batch_id)

    # 检查 Bastion 状态 (state.bastion_status 是 single source of truth)
    if state.bastion_status == "disconnected":
        state = "waiting_otp"
        按 §7 节奏发 OTP 请求卡片, 等 OTP / 用户 stop
        OTP 收到 → 同步重启 daemon → mark_connected() → state = "running"
```

```

# 检查飞书指令（每轮 Stage 5 后一次）
commands = runner.check_commands(feishu, state_path)
处理优先级 stop > pause > change_config > resume

# 检查终止条件
if pending_count == 0: → completed → break
if stop 指令: → stopped → break
if pause / consecutive_failures>50 / error_rate>0.8:
    state = "paused" → 等用户继续

# 进度通知
发送 progress / alert / milestone 卡片

```

清理: stop_heartbeat() + 写终态 + 完成卡片附 auto-fix commit 列表

Kanban 模式 (`kanban.enabled: true`) ——ut-supervisor 不跑 Stage 逻辑，只做监控:

```

state = "running"

# 启动前 3 个 Gateway 必须全 alive (systemd 独立管理)
gateways = runner.check_gateways_alive() # {ut-orchestrator: True, ...}
if not all(gateways.values()):
    缺失的 profile 飞书发红色卡片 → 退出

runner.create_initial_kanban_task(workflow_yaml) # 创建首个 ut-orchestrator 任务

while True:
    # 每轮 Gateway 健康检查
    gateways = runner.check_gateways_alive()
    if not all(gateways.values()):
        发 gateway_down 卡片 + 等 systemd 自动恢复 (轮询 30s)
        continue

    # 监控 Kanban 状态 (不读/写 manifest, 那是 ut-orchestrator subprocess 的活)
    stats = runner.poll_kanban_stats(board_slug)
    if stats["pending"] == 0 and stats["running"] == 0:
        → completed → break

    # 检查 Bastion 状态 (同线性, daemon 由 ut-supervisor 维护)
    # 检查飞书指令 (同线性)
    # 进度通知 (从 manifest.json 读 statistics, 不修改)

    sleep(30)

```

清理: stop_heartbeat() + 写终态 (不动 Gateway) + 完成卡片附 auto-fix commit 列表

14.4 Bastion 断联当前 batch 处理

Stage 3 Worker (ut-executor subprocess 或线性模式 supervisor 内)

检测 Bastion 错误 (agent.py run 连接失败)

- 调 `bastion.mark_disconnected()` 上报观察结果
- Worker 返回 `next_action=wait`, 不标记测试

ut-supervisor 看到 `state.bastion_status=disconnected`

- 当前 `iteration` 不写 `batch_results.json`
- 测试状态保持 `pending/原状态`
- 进入 `waiting_otp` (按 §7 渐进重发)

OTP 恢复后

- 同步 `daemon` 重启 → `mark_connected()`
- 下一 `iteration` → Stage 2 重新选 batch

心跳线程: 定期 ping bastion, 检测到断联 → 调 `mark_disconnected()`; 检测到恢复 → 调 `mark_connected()`。

心跳是辅助加速发现, **不直接发 OTP 卡片或切状态**——一切状态唯一入口是 ut-supervisor 主循环。

Kanban 模式说明: 所有 Worker subprocess (ut-executor 等) 通过 `agent.py run` 时复用 ut-supervisor 维护的 `daemon` 连接 (不重新走 OTP)。

14.5 Bastion 恢复后的 workflow 继续

OTP 恢复 + `daemon` 重启成功后状态机回到 `running`, workflow 自动从 **Stage 2 重新选 batch** 继续:

iteration N (被 Bastion 断联打断):

Stage 2: 选了 batch [`test_a`, `test_b`, `test_c`] → `batch_config.json`

Stage 3: 跑 `pytest` 时 Bastion 断了

worker 调 `bastion.mark_disconnected()` + 返回 `next_action=wait`

不写 `batch_results.json`

`test_a/b/c` 的 `manifest` 状态保持原样

[ut-supervisor 切 `waiting_otp` → OTP 收到 → `daemon` 重启成功 → 回 `running`]

iteration N+1 (恢复后):

Stage 2: 重新选 batch

`test_a/b/c` 状态还是 `pending` (`manifest` 没改) → 再次被选中

Stage 3: 重新跑 (这次 Bastion 在线)

Stage 4/5: 正常处理

为什么不会丢测试: `manifest` 是 single source of truth, **只在 Stage 5 (manifest-updater) 才写**。Stage 3 失败时 Worker 不写 `batch_results.json` → Stage 5 没有输入 → `manifest` 不更新该批 → 测试状态保持原样 → 下一轮 `batch-selector` 重新选中。

Kanban 模式下的恢复行为: ut-executor Worker subprocess 检测到 Bastion 错误时:

- 调 `bastion.mark_disconnected()` 上报到 ut-supervisor (共享 state)
- Worker subprocess 任务标记 `failed` 后退出
- Kanban 任务依赖链断裂 → ut-orchestrator 下一轮 (`depends_on=executor task`) 观察到 `executor` 失败
- ut-orchestrator 不更新 `manifest` 该批 → 重新创建 batch task (包含同一批未跑完的测试)

- 恢复后 Gateway[ut-executor] 自动派发新的 batch task → Worker 起来跑 → 正常推进

ut-supervisor 主循环只需切到 waiting_otp 走 OTP 流程即可；Kanban 任务层的失败/重建由 ut-orchestrator 在 Kanban 层自动消化，supervisor 不需要做额外协调。

iteration 计数器： workflow_state.iteration 在每轮 Stage 2 开始时递增，被打断的 iteration N 也算"跑过"（即便没产出 batch_results.json）。这只影响"我跑了 N 轮"的统计数字，不影响测试覆盖。

15. 完成处理

不自动归档。完成时只：

1. 写最终 manifest.json 和 workflow_state.json (status=completed)
2. 飞书发完成卡片（含统计；附带 git log master..2.5.1_ut_verify --oneline 摘要）
3. current_run.json 标记 status=completed
4. 不动 batch 目录、raw_log.txt、Gateway 进程

未来如需报告，写独立脚本 generate_report.py，用户手动调。

16. Worker SKILL.md 修改清单

16.1 unit-test-executor/SKILL.md

- 删除 head -200 / head -100 截断
- pytest 跑完后远端只写 {remote_batch_dir}/raw_log.txt
- Worker 调远端 grep + tail raw_log.txt 命令，回传文本 (≤200KB)
- Worker 把回传文本本地落盘到 {local_batch_dir}/summary.txt
- 远端日志路径写入 batch_results.json 的 batch 层级 remote_log 字段（包含 raw_log_path，不包含 summary_path，summary 是本地概念）
- OOM 检测 → 标 retrievable_error + error_type=oom
- timeout 检测 → 标 retrievable_error + error_type=timeout
- collection error → 标 error
- Bastion 断联检测 → 调 bastion.mark_disconnected() 上报，返回 next_action=wait，不标记测试
- 删除任何 Worker 自重试逻辑

16.2 failure-handler/SKILL.md

- 处理范围：failed + error（不处理 retrievable_error）
- 通过 test 的 last_batch_id resolve 到 batch_results.json 取 remote_log 路径
- 默认只读本地 summary.txt；不够时按需 agent.py run "tail -N <remote_log.raw_log_path>"
- 删除"行数 ≤5"/"架构级问题暂停"/"必须有补丁才修"等人为限制
- vLLM 源码修复约束：
 - 必须在分支 2.5.1_ut_verify 上 commit

- 启动前校验 HEAD 在该分支，不在则报错退出
- commit message 前缀 [auto-fix]
- C/E/D/P/M/S 处理细节：保留现有

16.3 batch-selector/SKILL.md + generate_batch.py

完整选取规则：

```
def select_batch(manifest, batch_size: int) -> list[Test]:
    selectable = [
        t for t in manifest["tests"] if
            (t["status"] == "pending") or
            (t["status"] == "fixed_pending_verify") or
            (t["status"] == "retriable_error" and t["retry_count"] < t["max_retry"]) or
            (t["status"] == "failed" and t["retry_count"] < t["max_retry"])
    ]
    # 优先级排序: pending > fixed_pending_verify > retriable_error > failed
    selectable.sort(key=lambda t: STATUS_PRIORITY[t["status"]])
    return selectable[:batch_size]
```

status × 选取规则：

status	选取?	条件	备注
pending	✓	无	新测试，优先级 1
fixed_pending_verify	✓	无	Stage 4 修复后等待验证，优先级 2
retriable_error	✓	retry_count < max_retry	OOM/timeout/网络抖动，优先级 3
failed	✓	retry_count < max_retry	断言失败重试（避免环境抖动误判），优先级 4
error	✗	—	不选，直接进 Stage 4
running	✗	—	上一轮未退出干净的脏数据，不应出现
passed	✗	—	终态
ignored	✗	—	终态

为什么 error 不进 batch-selector：

error 表示 collection error / import error 等“非测试代码本身”问题——重跑无意义（当前环境有结构性问题）。直接交 Stage 4 failure-handler 分析（C/E/D/P/M/S 分类）：

- failure-handler 修复成功 → 状态变 fixed_pending_verify → 下轮 batch-selector 重选验证
- failure-handler 判断不修（如平台不支持）→ 状态变 ignored（终态）

如果让 error 也进 batch-selector 重跑，会浪费整个 batch 时间，且大概率继续 error。

为什么 `retriable_error` 不进 `failure-handler`：

`retriable_error` 的产生原因（OOM/timeout/网络抖动）多为**环境性偶发**，重跑大概率能过；进 `failure-handler` 让 Agent 分析无意义（Agent 看到 OOM traceback 也只能建议重跑）。走 `batch-selector` 重选 `retry` 路径更直接。
`retry_count` 累加由 `manifest-updater`（Stage 5）负责，每轮 Stage 5 后 `retry_count++`。下次 Stage 2 `batch-selector` 看到 `retry_count >= max_retry` 时不再选取，由 `manifest-updater` 直接将其标记 `ignored`（理由 "max retry exceeded for {error_type}"，详见 §5.4）。

`batch_size` 行为：

场景	行为
<code>len(selectable) >= batch_size</code>	返回前 <code>batch_size</code> 个（按优先级排序）
<code>0 < len(selectable) < batch_size</code>	返回 partial batch（不补齐、不阻塞）
<code>len(selectable) == 0</code>	返回空列表 → supervisor 主循环看到 → 检查 <code>pending_count==0</code> → completed

优先级排序的作用：

1. pending

← 新测试优先（保证主线推进）
2. fixed_pending_verify

← 修复结果尽快验证（避免修复堆积）
3. retriable_error

← 偶发故障重试
4. failed

← 断言失败重试（最低优先级）

数字越小越优先。这样新测试一直在前进、修复结果尽快闭环、重试类排后面避免长期占用资源。

`generate_batch.py` 输出格式：

```
// {local_batch_dir}/batch_config.json
{
  "batch_id": "batch_20260619_001",
  "iteration": 42,
  "selected_count": 8,
  "tests": [
    {"test_id": "test_a", "selected_reason": "pending"},
    {"test_id": "test_b", "selected_reason": "retriable_error retry 1/3"},
    {"test_id": "test_c", "selected_reason": "fixed_pending_verify"}
  ]
}
```

`selected_reason` 字段记录"为什么被选中"，便于事后排查（不写 `manifest`，只写 `batch_config.json`）。

Kanban 模式下的执行位置：

`batch-selector` 的逻辑在 Kanban 模式下由 **ut-orchestrator profile 的 Worker subprocess** 执行（与 `manifest-updater` 在同一 Worker 内串行执行——先 `update_status` 后 `select_batch`，详见 §16.6）。线性模式下由 `ut-supervisor` 主循环直接执行。

16.4 manifest-updater/SKILL.md

- retry_count 累加
- 每个 test 更新 last_batch_id 字段为本轮 batch_id
- retrieable_error + retry_count >= max_retry → 直接 ignored
- statistics 加 retrieable_error 计数

16.5 manifest_schema.json

- status 枚举加 retrieable_error
- error_type 枚举加 oom / timeout
- tests[i] 加 last_batch_id 字段 (string, optional, pointer 到 batch_results.json)
- tests[i] 加 max_retry 字段 (integer ≥ 0, 默认从 workflow.yaml:max_retry_per_test 读取)
- 不在 tests[i] 加 remote_log —— 该字段在 batch_results.json 的 batch 层级

16.6 ut-orchestrator profile SOUL.md (Kanban 模式专属)

- profile 加载两份 SKILL: batch-selector/SKILL.md + manifest-updater/SKILL.md
- Worker subprocess 启动后, 依次执行:
 1. 读上一轮的 batch_results.json + handled_tests.json
 2. 跑 manifest-updater 逻辑 → 更新 manifest.json
 3. 检查 pending_count, 无 pending → 标记 workflow 完成 (写 done 状态)
 4. 否则跑 batch-selector 逻辑 → 创建新 batch task (assignee=ut-executor) + fix task (assignee=ut-fixer, depends_on=executor task) + 下一轮 orchestrator task (depends_on=fixer task)

17. 文件变更清单

文件	操作	说明
skills/ut/hermes_workflow/SKILL.md	 新建	Hermes Agent 指令 (通道差异层)
skills/ut/workflow_loop_core/SKILL.md	 新建	共享循环主体 (两个 Supervisor 都加载)
skills/ut/hermes_workflow/profile.yaml	 新建	ut-supervisor profile 配置 (飞书订阅 + 加载 hermes_workflow skill)
docs/guides/hermes-supervisor-service.md	 新建	ut-supervisor systemd unit 部署指南 (hermes-agent@ut-supervisor)

文件	操作	说明
<code>docs/guides/hermes-gateway-service.md</code>	 新建	3 Gateway systemd template unit 部署指南 (hermes-gateway@.service 实例化为 ut-orchestrator/ut-executor/ut-fixer)
<code>skills/ut/workflow/SKILL.md</code>	 更新	启动时一次性加载 4 份 Worker SKILL; 引用缺失时 reload; 调 <code>workflow_loop_core</code>
<code>skills/ut/workflow/scripts/hermes_runner.py</code>	 重构	工具模块定位, 删除 <code>stage_*</code> 函数; 删 <code>start_gateway</code> 加 <code>check_gateways_alive</code>
<code>skills/ut/workflow/workflow_state_schema.json</code>	 更新	<code>pending_config</code> 字段; 终态 <code>stopped/failed/completed</code> ; 删 <code>reconnecting</code>
<code>skills/ut/unit-test-executor/SKILL.md</code>	 更新	远端只写 <code>raw_log</code> , 本地写 <code>summary</code> ; Worker 不重试; <code>retriable_error</code> 标记; Bastion 上报
<code>skills/ut/unit-test-executor/scripts/execute_batch.py</code>	 更新	远端 <code>raw_log</code> 写盘 + Worker 远端 <code>grep/tail</code> + 本地 <code>summary</code> 落盘 + <code>batch_results.remote_log</code> 字段
<code>skills/ut/failure-handler/SKILL.md</code>	 更新	删除修复限制、vLLM 分支约束、不处理 <code>retriable_error</code> 、按需读 remote <code>raw_log</code>
<code>skills/ut/batch-selector/SKILL.md</code>	 更新	选取条件加 <code>retriable_error</code>
<code>skills/ut/batch-selector/scripts/generate_batch.py</code>	 更新	同上
<code>skills/ut/manifest-updater/SKILL.md</code>	 更新	<code>retriable_error</code> retry 用完 → ignored; 维护 <code>last_batch_id</code>
<code>skills/ut/shared/manifest_schema.json</code>	 更新	<code>status</code> 加 <code>retriable_error</code> ; <code>error_type</code> 加 <code>oom/timeout</code> ; <code>tests[i]</code> 加 <code>last_batch_id</code> + <code>max_retry</code>
<code>~/AppData/Local/hermes/profiles/ut-orchestrator/SOUL.md</code>	 更新	加载 <code>batch-selector</code> + <code>manifest-updater</code> 两份 SKILL (兼任 Stage 2+5)
<code>.agents/workflow.yaml</code>	 更新	<code>batch_size</code> 默认 8; <code>max_retry_per_test</code> 默认 3

18. 非目标

- 不修改 `gpu_scheduler.py` / `retry_test.py` 等已有脚本的功能
 - 不在 `hermes_workflow/` / `workflow_loop_core/` 下新增脚本文件
 - 不改变 `manifest.json` / `batch_config.json` 等数据格式（只扩展枚举、加 `last_batch_id` 指针）
 - 不改变现有飞书通知卡片格式（除完成卡片附加 `auto-fix commit` 列表）
 - 不做复杂的 `batch_size` 动态升降档规则表
 - **不引入 ut-bastion Worker**（第一阶段统一在 Supervisor 管 Bastion）
 - **不引入测试文件级粒度**（`test list` 仍要求每行一个具体用例 `file::test_name`）
 - 不做 `workflow` 完成后的自动归档/清理
 - 不在 `workflow` 内启动 Gateway——3 个 Gateway 由 `systemd` 独立服务管理
 - 不新增第 4 个 Kanban Worker profile（Stage 5 由 `ut-orchestrator` 兼任）
-

19. 演进项（不在第一阶段）

- `ut-bastion Worker`：等 Hermes 长期 Worker 能力确认 + Worker 飞书订阅能力确认后引入
 - 测试文件级粒度：`test list` 支持文件路径，初始化时调 `pytest --collect-only` 展开
 - 完成报告生成：`generate_report.py` 独立脚本
 - 针对 OOM 测试的“批 size 自适应降档”修复策略
 - `prompt cache` 实验验证：跑 N 轮统计实际 token cost
 - `ut-orchestrator` 进一步拆分：Stage 5 单独 `ut-updater` profile（如果后续发现“reconcile + schedule”糅合带来运维复杂度）
-

20. 修订历史

- **v1 (2026-06-18)**: 初稿（双通道双 skill）
- **v2 (2026-06-18)**: 加状态机、`pending_config`、Worker 链式加载
- **v3 (2026-06-18)**: 完整 grilling 后修订
- **v4 (2026-06-19)**: 二次 grilling 后修订（共享 `loop_core` / Bastion 检测策略 / SKILL 一次加载 + fallback / 远端日志 / vLLM 分支约束 / OTP 渐进重发 / 删 reconnecting / Gateway `systemd` / `max_retry` / `batch_size` 理由）
- **v5 (2026-06-19)**: 进程模型澄清——
 - 新增 §14.0 进程模型与部署架构 + 两张交互图（线性 + Kanban）
 - 明确 `ut-supervisor` 是 1 个长期 Hermes **Agent** profile（订阅飞书）
 - 明确 3 个 Kanban Gateway 是 1:1 绑 `ut-orchestrator/ut-executor/ut-fixer` profile
 - `check_gateway_alive()` → `check_gateways_alive()` 返回 dict
 - Kanban 模式 Stage 5 归属：**ut-orchestrator 兼 Stage 2+5**（manifest 写者唯一，避免并发）
 - `ut-supervisor` 在 Kanban 模式下不碰 manifest，只做监控/飞书/Bastion/终止条件
 - 远端日志策略再修正：**远端只写 raw_log.txt, summary.txt 仅本地存在**（删

remote_batch_dir/summary.txt 和 summary_path 字段)

- 新增文件: hermes_workflow/profile.yaml、hermes-supervisor-service.md、hermes-gateway-service.md、ut-orchestrator/SOUL.md 更新 (Kanban 模式加载 batch-selector + manifest-updater 两份 SKILL)